

ОБРАБОТКА ИСКЛЮЧЕНИЙ

Семинар 11

Обработка ошибок в C++

Существует два способа сообщить об ошибке, возникшей при выполнении функции: возвращение из функции кода ошибки и использование исключений.

Некоторые ошибки времени выполнения можно обнаружить заранее с помощью проверок в коде. Обычно, если ошибка обнаружена, то дальнейшее выполнение функции не имеет смысла, и нужно сообщить об ошибке в то место кода, откуда эта функция была вызвана. Для этого предназначен механизм исключений.

Ключевые слова

Исключения в языке C++ реализованы с помощью трех ключевых слов, которые работают в связке друг с другом: **throw**, **try** и **catch**.

Генерация исключений

В C++ оператор `throw` используется для сигнализирования о возникновении исключения или ошибки. Сигнализирование о том, что произошло исключение, называется генерацией исключения (или «выбрасыванием исключения»).

```
1  throw -1; // генерация исключения типа int
2  throw ENUM_INVALID_INDEX; // генерация исключения типа enum
3  throw "Can not take square root of negative number"; // генерация исключения типа const char* (строка C-style)
4  throw dX; // генерация исключения типа double (переменная типа double, которая была определена ранее)
5  throw MyException("Fatal Error"); // генерация исключения с использованием объекта класса MyException
```

Поиск исключений

В C++ используется ключевое слово `try` для определения блока стейтментов (так называемого «блока `try`»). Блок `try` действует как наблюдатель в поисках исключений, которые были выброшены каким-либо из операторов в этом же блоке.

```
1  try
2  {
3      // Здесь мы пишем стейтменты, которые будут генерировать следующее исключение
4      throw -1; // типичный стейтмент throw
5  }
```

Обработка исключений

Ключевое слово `catch` используется для определения блока кода (так называемого «блока `catch`»), который обрабатывает исключения определенного типа данных.

```
1  catch (int a)
2  {
3      // Обрабатываем исключение типа int
4      std::cerr << "We caught an int exception with value" << a << '\n';
5  }
```

Обработка исключений

```
6      try
7      {
8          // Здесь мы пишем стейтменты, которые будут генерировать следующее исключение
9          throw -1; // типичный стейтмент throw
10     }
11     catch (int a)
12     {
13         // Любые исключения типа int, сгенерированные в блоке try, приведенном выше, обрабатываются здесь
14         std::cerr << "We caught an int exception with value: " << a << '\n';
15     }
16     catch (double) // мы не указываем имя переменной, так как в этом нет надобности (мы её нигде в блоке не используем)
17     {
18         // Любые исключения типа double, сгенерированные в блоке try, приведенном выше, обрабатываются здесь
19         std::cerr << "We caught an exception of type double" << '\n';
20     }
21     catch (const std::string &str) // ловим исключения по константной ссылке
22     {
23         // Любые исключения типа std::string, сгенерированные внутри блока try, приведенном выше, обрабатываются здесь
24         std::cerr << "We caught an exception of type std::string" << '\n';
25     }
```

Обработчики всех типов исключений

Язык C++ предоставляет механизм обнаружения/обработки всех типов исключений — обработчик `catch-all`. Обработчик `catch-all` работает так же, как и обычный блок `catch`, за исключением того, что вместо обработки исключений определенного типа данных, он использует эллипсис (...) в качестве типа данных.

Обработчик `catch-all` должен находиться последним в цепочке блоков `catch`. Это делается для того, чтобы исключения сначала могли быть пойманы обработчиками `catch`, адаптированными к конкретным типам данных (если они вообще существуют).

Обработчики всех типов исключений

```
5      try
6      {
7          throw 7; // выбрасывается исключение типа int
8      }
9      catch (double a)
10     {
11         std::cout << "We caught an exception of type double: " << a << '\n';
12     }
13     catch (...) // обработчик catch-all
14     {
15         std::cout << "We caught an exception of an undetermined type!\n";
16     }
```

```
1      catch(...) {} // игнорируются любые непредвиденные исключения
```

Классы-исключения

Одной из основных проблем использования фундаментальных типов данных (например, типа `int`) в качестве типов исключений является то, что они, по своей сути, являются неопределенными. Еще более серьезной проблемой является неоднозначность того, что означает исключение, когда в блоке `try` имеется несколько `стейтментов` или вызовов функций.

Классы-исключения

Одним из способов решения этой проблемы является использование классов-исключений. Класс-исключение — это обычный класс, который выбрасывается в качестве исключения.

```
1  class ArrayException
2  {
3  private:
4      std::string m_error;
5
6  public:
7      ArrayException(std::string error) : m_error(error) {}
8      const char* getError() { return m_error.c_str(); }
9  };
```

Исключения и наследование

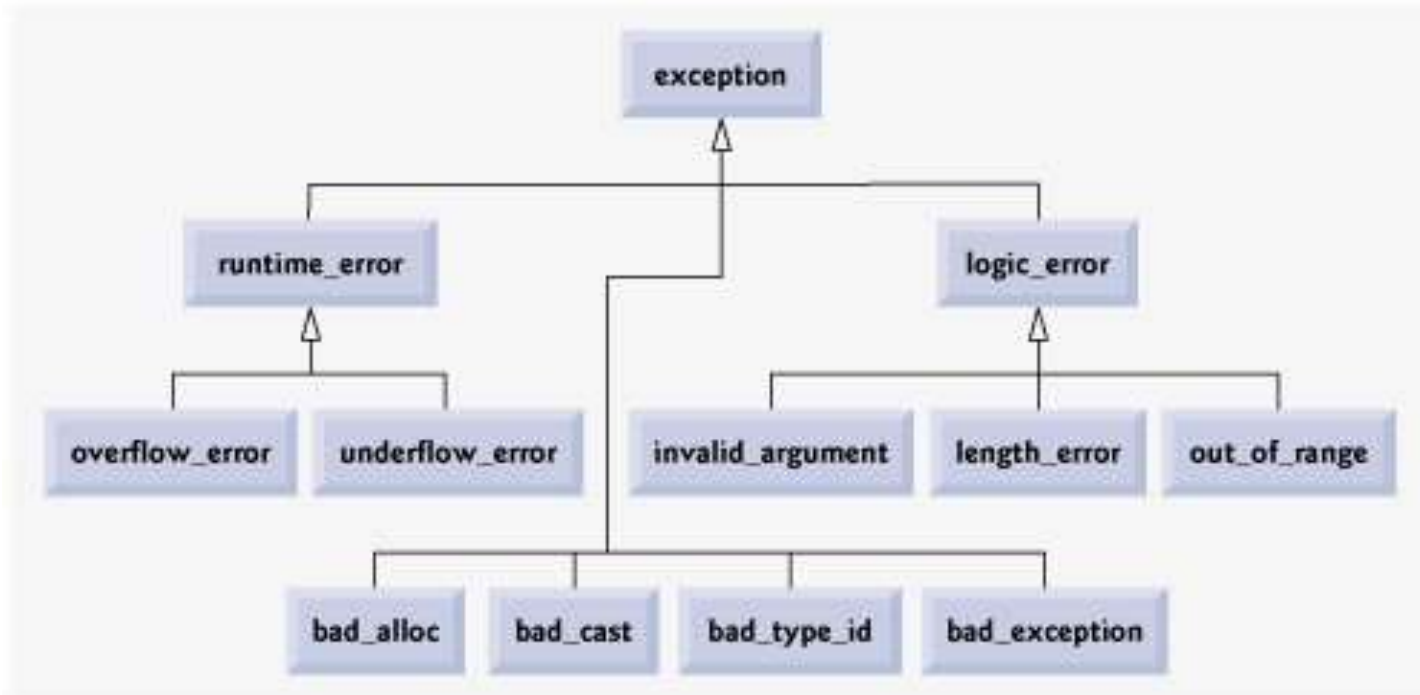
Обработчики могут обрабатывать исключения не только одного определенного класса, но и исключения дочерних ему классов.

Правило: Обработчики исключений дочерних классов должны находиться перед обработчиками исключений родительского класса.

std::exception

- **std::exception** — это небольшой интерфейсный класс, который используется в качестве родительского класса для любого исключения, которое выбрасывается в Стандартной библиотеке C++.
- в **std::exception** есть виртуальный метод `what()`, который возвращает строку C-style с описанием исключения.
- **std::exception** не генерируется напрямую, но вы можете генерировать исключения наследники
- можно создавать свои исключения, дочерние **std::exception**

std::exception



Исключения

```
1  #include <iostream>
2  #include <vector>
3
4  int main(int argc, char** argv)
5  {
6      std::vector<int> vec = {3, -5, 6, 7};
7      // do smth
8
9      // at() выработает исключение std::out_of_range if !(pos < size())
10     std::cout << vec.at(6) << std::endl;
11
12     return 0;
13 }
```

shulginaali@075-AShulgina:~/cpp\$./a.out

terminate called after throwing an instance of 'std::out_of_range'

what(): vector::_M_range_check: __n (which is 6) >= this->size() (which is 4)

Aborted (core dumped)

Исключения

```
1  #include <iostream>
2  #include <vector>
3
4  int main(int argc, char** argv)
5  {
6      std::vector<int> vec = {3, -5, 6, 7};
7      // do smth
8
9      try
10     {
11         // at() выработает исключение std::out_of_range if !(pos < size())
12         std::cout << vec.at(6) << std::endl;
13     }
14     catch (std::out_of_range const& exc)
15     {
16         std::cout << exc.what() << '\n';
17     }
18
19     return 0;
20 }
```

shulginaali@075-AShulgina:~/cpp\$./a.out

vector::_M_range_check: __n (which is 6) >= this->size() (which is 4)

shulginaali@075-AShulgina:~/cpp\$

Спецификатор noexcept

Спецификатор noexcept определяет функцию как не вызывающую исключения:

```
void doSomething() noexcept; // эта функция не выбрасывает  
исключения
```

На самом деле не мешает функции генерировать исключения или вызывать другие функции, которые потенциально могут генерировать исключения.

При выбросе исключения из noexcept-функции будет вызвана системная функция terminate()

Спецификатор noexcept является частью объявления функции, но не может использоваться для перегрузки

Задача

Напишите класс `Fraction`, конструктор которого принимает числитель и знаменатель.

Если пользователь передал в качестве знаменателя 0, то выбрасывайте исключение типа `std::runtime_error` (которое находится в заголовочном файле `stdexcept`).

В функции `main()` пользователь вводит два целых числа. Если числа, которые ввел пользователь, корректные, то выводится результат деления чисел.

Если же числа недопустимые, то должно обрабатываться исключение типа `std::exception` и выводиться его сообщение